🎓 Fraud Detection Intelligence Platform — Complete Project Brief

PROJ-CSD782 · Project-I · B.Tech. CSE-Data Science, Sem VII, Section A, Group 1
Team: Dhrubojyoti Hazra (Lead) · Isheeka Mukhopadhyay · Samprikta Dey · Adhiraj Das · Tiyasha Baidya
Supervisor: Rudranath Mitra
Title: *Fraud Detection in Digital Payment Systems Using Machine Learning*

1. Problem Statement (Official)

> *"Build a fraud detection framework capable of analyzing large volumes of digital transaction data stored in a data warehouse. Machine learning and anomaly detection techniques identify suspicious transactions in real time. The system generates risk scores and alerts to help financial institutions prevent fraudulent activities."*

Requirement → Our Solution Mapping

Requirement from Problem Statement	What We Build
"fraud detection framework"	End-to-end pipeline: ingestion → feature engineering → scoring → decision → feedback loop
"large volumes of digital transaction data"	PaySim dataset: **6.3 million** digital payment transactions (TRANSFER, CASH_OUT, PAYMENT, DEBIT)
"stored in a data warehouse"	**Supabase (Postgres)** for live data + **DuckDB** as an analytical data warehouse for aggregation & historical queries
"machine learning"	**XGBoost** (supervised classifier for known fraud patterns)
"anomaly detection techniques"	**Autoencoder** (unsupervised neural network — detects novel/zero-day fraud via reconstruction error)
"identify suspicious transactions in real time"	**FastAPI** scoring service + **Supabase Realtime** (Postgres logical replication) pushes instant updates

Requirement from Problem Statement	What We Build
"generates risk scores"	Calibrated combined score (0–1) from weighted aggregation of XGBoost probability + Autoencoder anomaly score
"alerts"	Escalation workflow: flagged transactions → REVIEW / ESCALATE queues → Analyst dashboards with SLA tracking
"help financial institutions"	Multi-branch banking simulation (5 branches), role-based dashboards (Analyst / Senior / Team Lead), RLS-enforced access
"prevent fraudulent activities"	Human-in-the-loop decision flow + feedback loop (analyst verdicts feed retraining)

Why This Problem Matters (India Context)

India's shift to UPI and card-based digital payments has outpaced legacy, rule-based fraud defenses. The core tradeoff:

Extreme	Consequence
Undetected fraud (low recall)	Direct financial losses
Over-aggressive detection (low precision)	Genuine customers blocked → revenue loss, churn

> **Important**
>
> With fraud typically under 1% of transactions, a model that predicts "no fraud" every time achieves 99%+ accuracy but catches zero fraud. The project must optimize for **recall** and **precision** on the minority (fraud) class — not just accuracy.

2. Dataset Strategy — Digital Payment Fraud

> **Note**
>
> Our project title is **"Fraud Detection in Digital Payment Systems"** — encompassing UPI, mobile money, online transfers, and digital wallets — NOT limited to credit card fraud. We use datasets that reflect diverse digital payment modes.

🥇 Primary Dataset: PaySim — Synthetic Financial Datasets for Fraud Detection

Property	Detail
Source	Kaggle – PaySim
Origin	Synthetic data generated from real mobile money transaction logs (African mobile money service, 14+ countries)
Total transactions	**6,362,620** (~6.3 million)
Simulation period	30 days (744 steps, 1 step = 1 hour)
Transaction types	`CASH-IN`, `CASH-OUT`, `DEBIT`, `PAYMENT`, `TRANSFER`
Fraud logic	Agents take control of accounts → transfer funds → cash out
Downloads	156,000+ (one of Kaggle's most popular fraud datasets)
License	CC BY-SA 4.0
Published research	PhD thesis by E.A. Lopez-Rojas (2016, European Modeling & Simulation Symposium)

Columns:

Column	Description
`step`	Time unit (1 step = 1 hour)
`type`	Transaction type: CASH-IN, CASH-OUT, DEBIT, PAYMENT, TRANSFER
`amount`	Transaction amount in local currency
`nameOrig`	Sender / originator customer ID
`oldbalanceOrg`	Sender balance before transaction
`newbalanceOrig`	Sender balance after transaction
`nameDest`	Recipient customer ID
`oldbalanceDest`	Recipient balance before transaction

Column	Description
`newbalanceDest`	Recipient balance after transaction
`isFraud`	**Target:** 1 = fraud, 0 = legitimate
`isFlaggedFraud`	Business rule flag (transfers > 200,000)

> **Tip**
>
> **Why PaySim is ideal for our project:**
>
> - Simulates **digital payment transactions** (transfers, cash-out, mobile money) — directly aligns with our "Digital Payment Systems" title
> - 6.3M transactions provides enough volume for robust ML training
> - **Interpretable features** (not PCA-anonymized) — we can engineer real features like velocity, balance anomalies, transaction type patterns
> - Fraud logic mimics real-world account-takeover → transfer → cash-out pattern
> - Published in academic research — citable in our report
> - Business rule flag (`isFlaggedFraud`) lets us benchmark our ML against naive rule-based detection

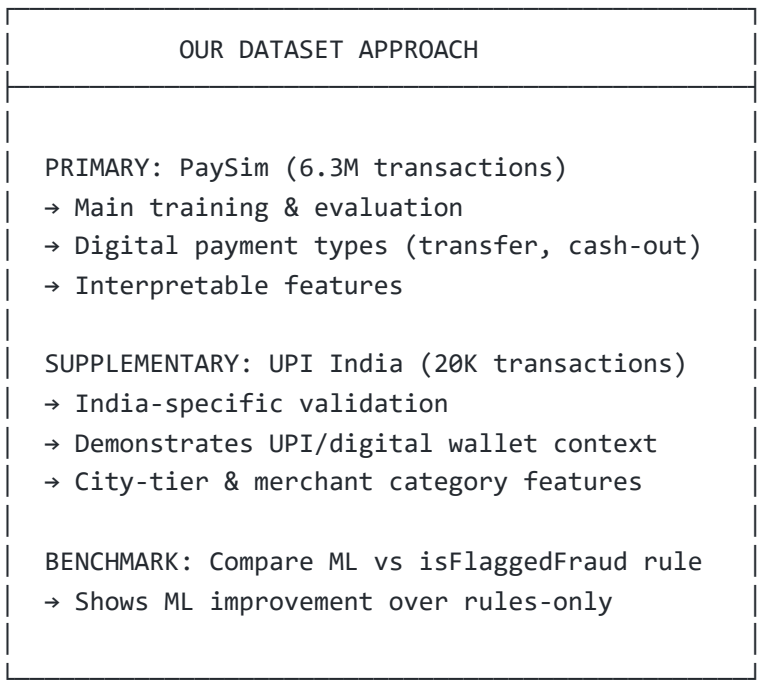
🥈 Supplementary Dataset: Online Payments Fraud Detection

Property	Detail
Source	Kaggle – Online Payments Fraud Detection
Total transactions	~6.3 million
Transaction types	Online payment types (PAYMENT, TRANSFER, CASH_OUT, DEBIT, CASH_IN)
Columns	step, type, amount, nameOrig, oldbalanceOrg, newbalanceOrig, nameDest, oldbalanceDest, newbalanceDest, isFraud
Downloads	33,800+
License	CC BY-NC-SA 4.0

🥉 India-Specific Context: UPI Payment Transactions India

Property	Detail
Source	Kaggle – UPI Payment Transactions India
Total transactions	20,000 (synthetic)
Key features	Fraud labels, city-tier distributions (Tier 1/2/3), ₹1L cap, transaction velocity, amount deviation scores
India-specific	City tiers, UPI payment modes, Indian merchant categories
Use case	Supplementary validation — demonstrates India-specific digital payment context

Multi-Dataset Strategy



> **Important**
>
> **Key advantage over using the Credit Card Fraud dataset:** PaySim has **interpretable features** (amount, balance, transaction type) rather than anonymized PCA components (V1-V28). This means we can engineer meaningful features (velocity spikes, balance anomalies, suspicious transfer→cash-out chains) and explain them to analysts — directly supporting our explainability story.

3. Industry Research — Three Real Systems Studied

We grounded our design by studying how three production fraud systems work:

3.1 Stripe Radar

- ML inside the card authorization path (<100ms latency constraint)
- Evolved from XGBoost → Wide&Deep → DNN (ResNeXt-inspired)
- Custom low-latency feature store (hardest engineering problem)
- "Risk Insights" for explainability (added because DNN is a black box)
- Cross-merchant network effects — their biggest edge

3.2 Razorpay Thirdwatch

- E-commerce fraud/RTO prevention (acquired 2019)
- "Forensic evidence" collection: device ID, IP, email, time-to-order
- ML + configurable rules — **hybrid approach**
- Central human decision point — analyst makes final call
- Kafka → Spark → Delta Lake/Hudi pipeline

3.3 Feedzai

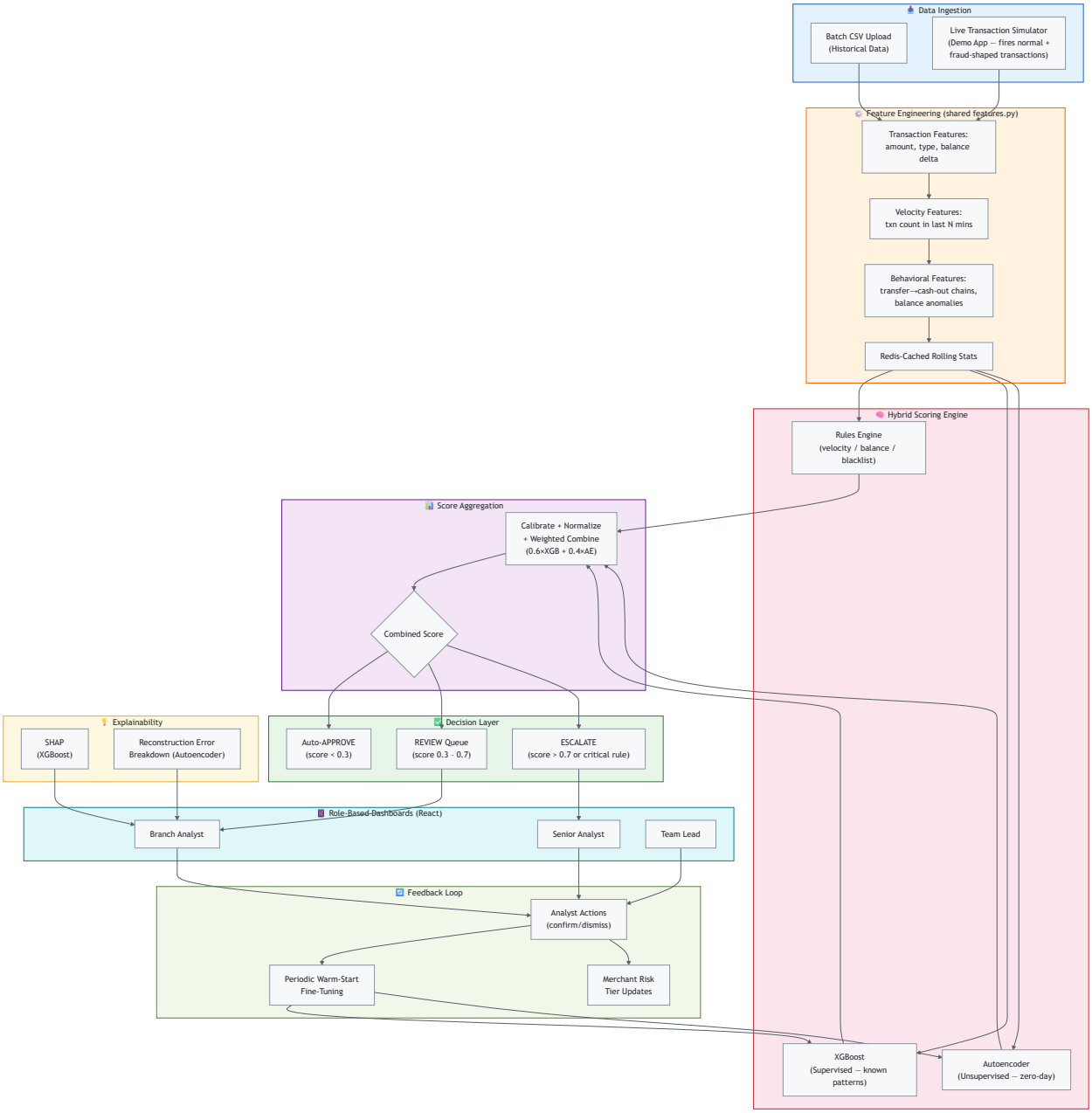
- Enterprise RiskOps for banks — most explainability-focused
- "Railgun" streaming engine + "Pulse" risk engine
- Rules + ML are **peers, not a fallback pair**
- "Whitebox Explanations" — plain-text rationale per decision (regulatory compliance)
- 4× more fraud caught, 50% fewer alerts vs. rules alone

Universal Pattern All Three Share

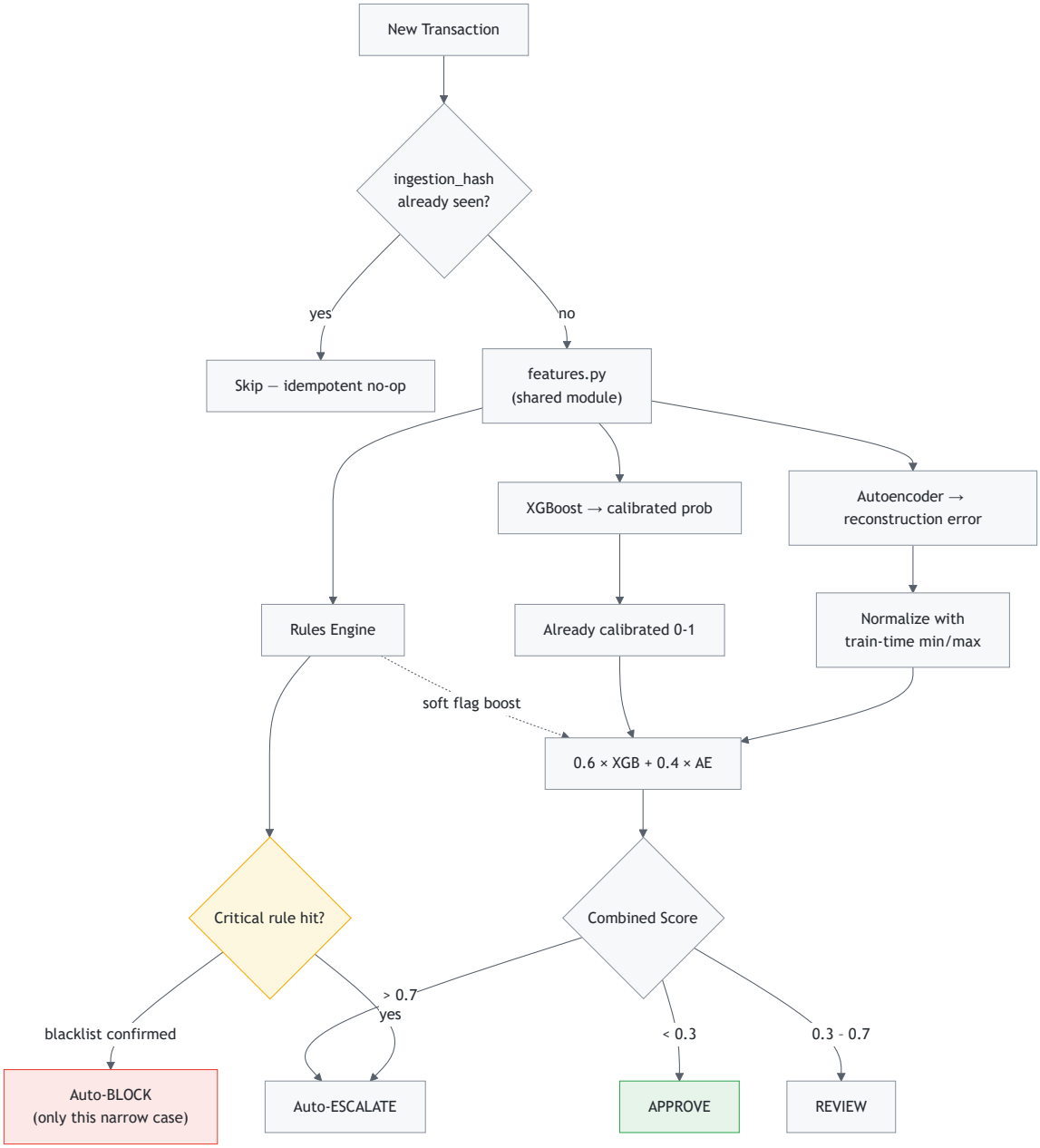
```
Ingestion → Feature Computation → Hybrid Scoring (Rules + ML)
    → Explainability → Decision + Human Override
    → Feedback Loop → Continuous Retraining
```

4. System Architecture

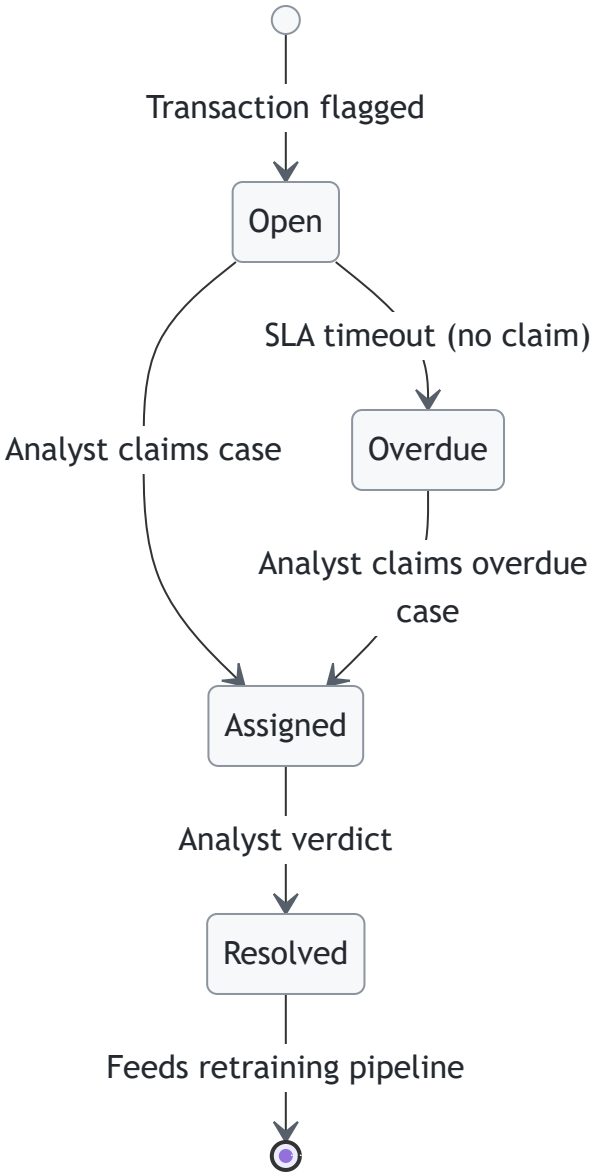
4.1 High-Level Architecture Diagram



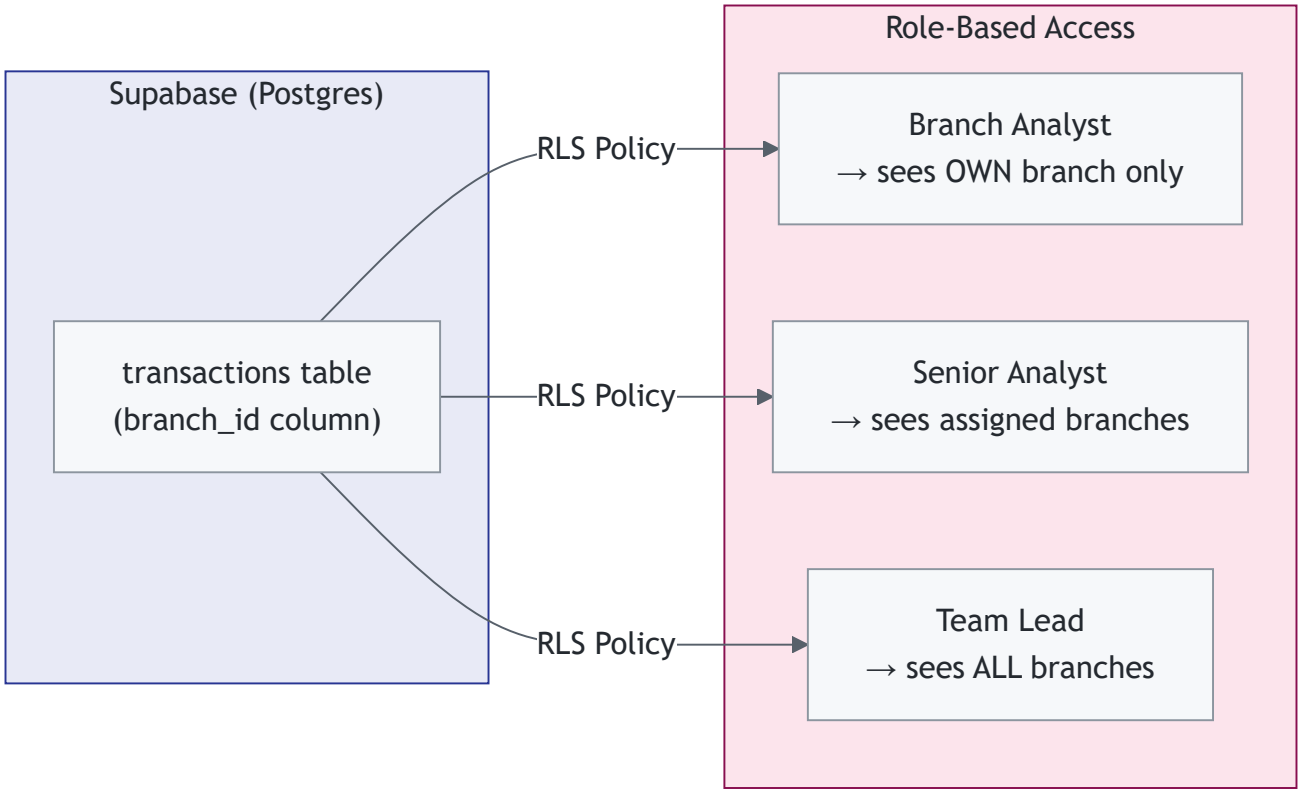
4.2 Score Aggregation Flow (Detailed)



4.3 Escalation State Machine



4.4 Multi-Branch Access Control (RLS)



5. Machine Learning Approach

Component	Choice	Role	Justification
Supervised model	XGBoost	Flags known fraud patterns	Best performer on tabular data at this scale
Unsupervised model	Autoencoder (neural net)	Zero-day / novel fraud detection	Learns "normal" patterns; novel fraud = high reconstruction error
Class imbalance	SMOTE + class weighting	Handle <1% fraud rate	Standard approach for severe imbalance
Explainability	SHAP + reconstruction error breakdown	Every flagged txn has a reason	Mirrors Stripe Risk Insights / Feedzai Whitebox

Why Autoencoder for Zero-Day Detection?

- Trained **only on legitimate transactions** → learns to reconstruct "normal"
- Novel fraud produces **high reconstruction error** — no fraud labels needed

- Small model (few thousand params) — trains in minutes on free Colab
- Supports **warm-start fine-tuning** (unlike Isolation Forest which needs full rebuild)
- Autoencoder architecture:

```
Input (features) → Dense(100) → ReLU → Dense(20) → ReLU  [encoder]
                 → Dense(100) → ReLU → Dense(N)          [decoder]
```

Feature Engineering (Advantage of PaySim over Credit Card dataset)

Since PaySim has **interpretable features**, we can engineer:

Feature Category	Examples
Transaction-level	Amount, type (TRANSFER vs PAYMENT), balance delta
Velocity	# of transactions in last 1h / 6h / 24h per account
Balance anomaly	Sudden balance drop, zero-balance-after patterns
Behavioral chains	TRANSFER → CASH_OUT sequence (classic fraud pattern)
Temporal	Hour-of-day, unusual timing patterns
Amount deviation	Z-score vs. account's historical average

> **Tip**
>
> PaySim's fraud agents specifically follow a **TRANSFER → CASH_OUT** pattern (take over account → transfer funds → cash out). This is a real-world digital payment fraud pattern that our rules engine can detect explicitly, and our ML can learn implicitly.

Retraining Strategy: "Periodic Warm-Start Fine-Tuning"

- NOT real-time online learning (unsolved research problem — catastrophic forgetting)
- Scheduled job loads saved weights → continues training on new confirmed-normal data → saves
- Honest, defensible claim without overclaiming

6. Tech Stack

Layer	Technology	Why
Backend API	**FastAPI** (Python)	Async, fast, ML-friendly
Database	**Supabase** (managed Postgres)	RLS, Realtime, Auth, Storage — all built-in
Real-time	**Supabase Realtime** (Postgres logical replication)	No Kafka overhead at our scale
Cache	**Redis**	Rolling stats for velocity features
ML	**scikit-learn, XGBoost, PyTorch**	Industry-standard for tabular ML
Explainability	**SHAP**	Per-feature importance for every prediction
Frontend	**React**	Role-based dashboards
Deployment	**Render/Railway** (API) + **Vercel** (frontend)	Free tier, real deployment

7. Team Responsibilities

Member	Role	Responsibilities
Dhrubojyoti Hazra (Lead)	Frontend + Escalation	Analyst/Senior/Team Lead dashboards, escalation UI
Isheeka Mukhopadhyay	ML Engineer	ML pipeline, model training, evaluation, versioning
Samprikta Dey	Backend Engineer	Scoring service, velocity cache, rules engine, real-time
Adhiraj Das	Explainability + Simulator	SHAP layer, live transaction simulator
Tiyasha Baidya	Data + DevOps	DB schema, RLS policies, batch pipeline, deployment

8. 14-Week Timeline

Weeks	Milestone	Risk Level
1–2	Schema + RLS + Auth roles	🟢 Low
3–4	Batch ingestion pipeline	🟢 Low
5–6	ML ensemble (XGBoost + Autoencoder) + SHAP	🟡 Medium
7–8	Live simulator + real-time wiring	🔴 **Highest risk** (started early)
9–10	Escalation workflow	🟡 Medium
11–12	Aggregate dashboard + analytics	🟢 Low
13–14	Reliability hardening + demo rehearsal	🟢 Low

9. Differentiation — Why This Isn't "Just Another ML Project"

What makes it different	Detail
Digital payment focus	PaySim (mobile money) + UPI context — not just credit cards
Hybrid scoring (Rules + ML)	Mirrors real production systems (Stripe, Razorpay, Feedzai)
Neural network for zero-day	Autoencoder detects novel fraud without labeled examples
Interpretable features	Real features (amount, type, velocity) — not anonymous PCA components
Explainability as first-class	SHAP + reconstruction error — every flag has a stated reason
Human-in-the-loop workflow	Analyst → Senior → Team Lead escalation with SLA tracking
Multi-branch with RLS	Real access control, not just UI-level filtering

What makes it different	Detail
Feedback loop	Analyst decisions feed back into model retraining
Grounded in industry research	Architecture derived from studying Stripe, Razorpay, Feedzai

10. Current Status

- [x] Review 1 (Synopsis/Proposal) — **completed and submitted**
- [x] Problem domain analysis
- [x] Study of 3 existing systems (Stripe, Razorpay, Feedzai)
- [x] Full system architecture designed and technically reviewed
- [x] ML approach finalized (XGBoost + Autoencoder)
- [x] Score aggregation logic designed
- [x] Gaps identified and resolved (idempotency, auto-block policy, SLA fallback)
- [ ] **Next:** Begin implementation — Week 1–2 milestone (schema + RLS + auth)

11. Key References

1. PaySim — E.A. Lopez-Rojas et al. (2016), European Modeling & Simulation Symposium
2. Fraud Detection Handbook — ULB ML Group (open source, Colab-runnable)
3. F1000Research — Hybrid AE + XGBoost framework
4. Stripe Engineering — How we built Radar
5. Feedzai — Pulse Risk Engine
6. Razorpay — Breaking the Risk Review Black Box
7. Kaggle — PaySim Dataset
8. Kaggle — UPI Payment Transactions India

All changes saved